AdAgentFlow-RT: A Contractual Runtime for Reliable High-Throughput Long-Horizon Multi-Agent Workflows

Anonymous Authors
Anonymous Institution

ABSTRACT

Existing agent benchmarks primarily measure whether an agent completes a task. This paper studies a complementary question: whether a contract-aware runtime remains bounded, diagnosable, and recoverable under concurrent production-style execution. We present AdAgentFlow-RT, a contractual runtime that represents workflows as contract-bearing execution graphs, monitors schema, semantic, dependency, resource, and recovery contracts, localizes faults, applies bounded recovery policies, and records event-sourced traces. We do not introduce a new task dataset. Instead, we build a production-stress evaluation harness on top of public τ^3-bench and AgentChangeBench fixtures and drive it with a live Qwen3-8B endpoint served by vLLM. The harness records fixture-backed task and policy scores while adding runtime-stability metrics such as dead-letter rate, retry amplification, explicit contract violations, over-rejection (false dead-letter), and auditability signals. In the live-LLM evaluation across 1,392 runs, the main τ^3-bench slice contributes 216 sampled executions per method over 36 distinct task-trial instances reused across concurrency and fault-rate cells. Schema-only validation slightly leads raw fixture success, while the full runtime remains comparable on cost-per-success and differentiates itself by making contract violations and recovery decisions explicit in the trace. The current main matrix does not show non-zero bounded-recovery success, and vanilla reports non-zero over-rejection, so both metrics are treated as audit signals rather than headline wins. The artifact includes the runtime kernel, production-stress harness, four runtime strategies, seven stressors, four ablations, JSONL traces, aggregated summaries, paper tables, regenerated figures, and an exploratory controlled fault-injection pipeline.

CCS CONCEPTS

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Runtime environments.*

KEYWORDS

multi-agent systems, runtime reliability, fault injection, runtime verification, agent benchmarks

1 INTRODUCTION

Agent systems increasingly combine planning, tool use, memory, verification, and multiple specialized components. Public benchmarks have improved the field's ability to measure task completion, but production deployments face a broader reliability problem: the runtime must bound cost, detect contract violations, avoid silent failures, contain faulty artifacts, recover without unbounded retry loops, and preserve traces that support diagnosis.

AdAgentFlow-RT reframes an existing multi-agent engineering prototype as a contractual runtime for reliable high-throughput multi-agent workflows. The prior advertising generation workflow remains an example domain plugin. The paper contribution is a reusable runtime abstraction and a stress evaluation harness that can wrap public task environments without changing their task data.

This work studies three research questions. First, can executable contracts over schemas, semantics, dependencies, budgets, and recovery policies turn malformed or off-policy agent outputs into explicit runtime events rather than implicit downstream corruption? Second, can bounded recovery policies avoid the retry amplification of retry-only execution while keeping recovery decisions inspectable under injected production faults? Third, can event-sourced traces make runtime failures auditable enough to support fault localization and reproducible post-hoc diagnosis?

The main matrix is run against the live Qwen3-8B endpoint served by vLLM 0.23.0 and the public τ^3-bench and AgentChangeBench task fixtures. The evidence does not support a simple "highest task-success" claim: schema-only validation slightly outperforms the full runtime on raw τ^3-bench task success (15.3% vs. 14.8%). Instead, the current result supports a narrower runtime claim: AdAgentFlow-RT keeps cost-per-success comparable to schema-only while making contract violations and recovery decisions inspectable in the trace; retry-only amplifies calls without producing runtime-level diagnosis. Across 1,392 live-LLM rows, all four configurations report zero measured silent failures, but vanilla still shows non-zero over-rejection (false dead-letter), so these metrics should be read as audit outcomes rather than standalone evidence that contracts are sufficient.

The key distinction from a benchmark paper is that the workload is not new. tau2-bench / τ^3-bench and AgentChangeBench provide public task environments and fixture metadata [2, 12]. AdAgentFlow-RT contributes the execution substrate around those tasks: contract enforcement, fault localization, bounded recovery, dead-letter handling, and trace-derived reliability metrics.

This paper makes the following contributions:

- A contractual execution graph abstraction for multi-step agent workflows, including contracts over artifacts, dependencies, resources, and recovery behavior.
- Executable contracts and bounded recovery semantics implemented in a runtime kernel with a scheduler, monitor, artifact store, fault localizer, recovery controller, and event-sourced trace.
- A production-stress harness that reuses public benchmark fixtures while running comparable runtime strategies under matched models, tools, fault distributions, concurrency, and recovery budgets.
- A trace-derived reliability metric suite that complements fixture-backed task success with dead-letter rate, recovery

success, retry amplification, explicit contract violations, and auditability signals.

The current artifact includes deterministic smoke checks for development and an audited external main matrix for tau2-bench / τ^3-bench and AgentChangeBench. The paper claims in Section 6 are generated from the external matrix rather than smoke rows.

2 BACKGROUND AND MOTIVATION

Multi-agent workflows fail in ways that are not captured by final task success alone. A workflow can complete while using stale context, propagating a contaminated artifact, violating a resource budget, silently accepting a malformed tool response, or retrying until cost becomes unacceptable.

Retry and prompt repair are useful but insufficient as the only reliability mechanism. They do not identify the responsible node, downstream contamination, recovery budget, or whether the runtime should rollback, reroute, degrade, escalate, or dead-letter a task.

2.1 Failure Modes

We focus on failures that appear at runtime boundaries:

- **Contract failures**: malformed outputs, schema drift, missing artifacts, failed preconditions, or semantic mismatch.
- **Coordination failures**: duplicate node execution, stale context, missing dependency artifacts, and inconsistent downstream state.
- **Resource failures**: latency, token, tool-call, or retry budgets exceeded during long-horizon execution.
- **Environment failures**: tool timeouts, rate limits, partial responses, and worker crashes.
- **Silent failures**: runtime success states that disagree with the fixture-backed task scorer.

These failures motivate runtime-level guarantees that are orthogonal to model capability. A stronger model may complete more tasks, but a production runtime still needs traceability, bounded recovery, and controlled failure containment.

2.2 Why Public Benchmarks Are Still Needed

The harness deliberately uses public benchmarks instead of creating a new dataset. Public tasks provide comparability and established task metadata. The current artifact uses fixture-backed assertion scoring rather than a full benchmark-native simulator rerun. The harness adds stress dimensions around the same task fixtures: concurrency, repeated trials, injected tool faults, schema drift, duplicate execution, stale context, and recovery budgets. This separation lets the paper ask a runtime question without weakening the benchmark question.

3 CONTRACTUAL EXECUTION MODEL

AdAgentFlow-RT models a multi-agent workflow as a Contractual Execution Graph. Nodes declare a role, capability, agent provider, and contract name. Edges declare artifact dependencies. Contracts define input and output schemas, preconditions, postconditions, semantic checks, resource budgets, recovery policies, and escalation policies.

The first implementation supports DAG execution, including sequential paths, fan-out, joins, fallback branches, and rollback targets. Each produced artifact carries provenance, validation status, and downstream consumers. This makes failure propagation explicit and measurable.

3.1 Contracts

A contract binds a runtime capability to observable obligations:

- **Schema contract**: validates inputs and outputs.
- **Semantic contract**: captures domain invariants such as policy compliance or goal alignment.
- **Dependency contract**: requires upstream artifacts to exist and be valid before a node runs.
- **Budget contract**: bounds latency, LLM calls, tool calls, tokens, and retries.
- **Recovery contract**: lists permitted recovery actions and escalation behavior.

The current implementation represents contracts as typed Python dataclasses and supports JSON-Schema validation with a dependency-light fallback validator for minimal environments.

3.2 Artifacts and Propagation

Every node produces zero or more artifacts. An artifact records its producer, contract name, content, validation status, provenance, and downstream consumers. When a contract violation occurs, the runtime localizes the responsible node and traverses the graph to estimate affected downstream nodes and contaminated artifacts. These quantities directly support the metrics *fault propagation depth* and *contaminated artifact count*.

3.3 Bounded Recovery

Recovery is selected from a bounded policy rather than an open-ended retry loop. The controller currently supports quick repair, retry, mock reroute, rollback target selection, downstream invalidation, degradation, human escalation, and dead-letter. A recovery decision records the selected action, target node, reason, budget cost, and whether execution may continue.

4 ADAGENTFLOW-RT RUNTIME

The runtime contains a contract registry, scheduler, monitor, artifact store, fault localizer, recovery controller, event log, and replay utilities. Before a node runs, the monitor checks dependency and precondition contracts. After execution, it checks schema and budget contracts and emits violations. The fault localizer maps violations into operational classes such as artifact, coordination, resource, tool-environment, runtime, and verifier faults. The recovery controller selects bounded actions such as quick repair, retry, reroute, rollback, invalidation, degradation, human escalation, or dead-letter.

Every event is traceable by task identifier and run identifier. Events include graph creation, contract checks, node scheduling, artifact production, violations, fault diagnoses, recovery decisions, checkpoints, rollbacks, dead letters, and task finalization.

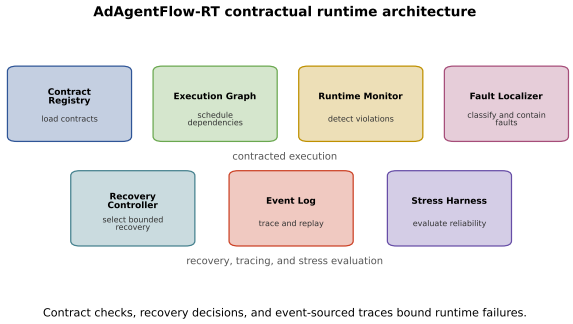

Figure 1: AdAgentFlow-RT contractual runtime components.

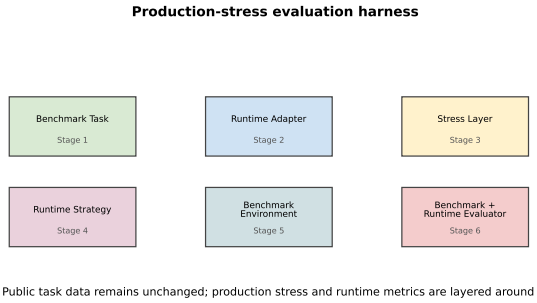

Figure 2: Production-stress harness layered around public benchmark tasks.

4.1 Runtime Kernel

Figure 1 shows the runtime components. The contract registry stores executable contract specifications. The graph planner constructs a dependency graph. The scheduler selects runnable nodes whose dependencies are satisfied. The monitor checks contracts before and after node execution. The artifact store records produced artifacts and validation status. The fault localizer maps violations into operational fault classes. The recovery controller selects bounded actions from policy and remaining budget. The event log records each runtime transition for replay and metric extraction.

4.2 Event-Sourced Trace

The event log records 'graph.created', 'contract.loaded', 'node.started', 'contract.checked', 'fault.injected', 'contract.violated', 'fault.localized', 'recovery.started', 'recovery.selected', 'recovery.succeeded', 'artifact.produced', 'dead_letter.created', and 'task.finalized'. Each event carries 'task_id', 'run_id', optional 'node_id', status, payload, and timestamp. The smoke harness also records relative millisecond offsets so trace metrics can estimate mean time to detect and recover.

4.3 Runtime Strategies

The harness implements four comparable strategies:

- **Vanilla**: directly executes the benchmark-style task without runtime contracts or recovery.
- **Retry-only**: retries after failure until a bounded retry count is exhausted.
- **Schema-only**: validates schema and applies repair/retry, but does not localize graph-level faults.
- **AdAgentFlow-RT**: applies contract monitoring, fault localization, bounded recovery, artifact invalidation hooks, dead-letter handling, and event trace.

5 PRODUCTION-STRESS EVALUATION HARNESS

The harness wraps public benchmark tasks without modifying their data. tau2-bench / τ^3-bench provide the main workload across airline, retail, and telecom domains. AgentChangeBench provides supplementary goal-change recovery evaluation across airline and retail domains.

We compare vanilla tool calling, retry-only execution, schema-only validation, and AdAgentFlow-RT. Stressors include tool timeouts, rate limits, partial responses, schema drift, stale context, duplicate messages, and worker crashes. All experiments write JSONL records with benchmark-fixture-backed scoring, runtime metrics, injected faults, events, and recovery outcomes. The current adapter preserves public task data and assertion fields, but it is not a replacement for a full benchmark-native simulator rerun; Section 6 reports this limitation explicitly.

Figure 2 summarizes the evaluation path. A benchmark task is passed to a runtime adapter, stressors perturb tool or context boundaries, a runtime strategy executes the task, and fixture-backed benchmark assertions plus runtime-native evaluators produce a unified JSONL record.

5.1 Benchmark Adapters

The tau2-bench / τ^3-bench adapter supports 'benchmark_repo_path', domain, task identifiers, number of tasks, trials, agent model, user model, and concurrency. If the external benchmark checkout is absent, the harness fails with a clear diagnostic in external mode or uses deterministic mock tasks in smoke mode. The AgentChangeBench adapter preserves the native metric slots TSR, TUE, TCRR, and GSRT.

5.2 Stressors

Stressors share a reproducible interface with a rate and seed. The implemented stressors simulate timeouts, rate limits, partial responses, schema drift, stale context, duplicate messages, and worker crashes. Each injected fault is recorded in the JSONL row and, for AdAgentFlow-RT, in the runtime trace.

5.3 Metrics

The harness reports task success, assertion coverage, and policy-compliance proxies derived from the public fixture metadata, plus runtime-stability metrics:

- throughput, p50/p95/p99 latency, dead-letter rate, and cost per successful task;
- recovery success rate, contract violation rate, retry amplification, extra tool calls, and extra LLM calls;
- silent failure rate, fault propagation depth, contaminated artifact count, mean time to detect, and mean time to recover when trace timestamps and artifact propagation data are available.

Every experiment writes machine-readable JSONL. Aggregation scripts produce JSON and CSV summaries, figure data, and paper-facing tables. Metrics that are undefined, such as cost per successful task when a method has zero successes, are emitted as missing values rather than zeros.

6 EXPERIMENTS

We evaluate four runtime strategies (vanilla tool calling, retry-only, schema-only, AdAgentFlow-RT) on three public benchmark task families (τ^3-bench *airline / retail / telecom* and AgentChangeBench *airline* / *retail*) under controlled concurrency, repeated trials, and injected faults. Every experiment writes a machine-readable JSONL row keyed by task, run, method, ablation, adapter mode, and concurrency.

Real benchmark data, real LLM. The main matrix uses public τ^3-bench and AgentChangeBench task fixtures cloned at their upstream GitHub SHAs. The runtime adapters drive a live Qwen3-8B endpoint served by vLLM 0.23.0 with constrained GPU memory so the harness can co-tenant on the shared GPU. Every run is recorded in external adapter mode; the customer goal, domain policy, and tool inventory are passed into the prompt, and public natural-language assertions drive fixture-backed scoring. This is not yet a full benchmark-native simulator execution, so we use the results as a production-stress harness study rather than a new benchmark leaderboard.

6.1 Main Matrix

The main setting samples from three public task pools and evaluates all four methods under a matched matrix:

- **Domains**: τ^3-bench *airline* (50 tasks), *retail* (114), *telecom* (2,285); AgentChangeBench *airline*, *retail* (50 tasks each).
- **Concurrency**: 1, 5.
- **Fault rates**: 0, 0.1, 0.2.
- **Stressors**: tool timeout, partial response, schema drift, stale context, duplicate message.
- **Tasks per cell**: 6, **trials**: 2.

The full matrix produces 1,392 JSONL rows across 39 runs (84 τ^3-bench summary rows + 32 AgentChangeBench summary rows = 116 total) in approximately four hours on a single Qwen3-8B endpoint. For the main τ^3-bench table, each method contributes 216 sampled executions over 36 distinct task-trial instances; the same task-trial instances are reused across multiple concurrency and fault-rate cells, so the 216 rows should not be interpreted as 216 independent tasks.

Table 1: Main τ^3-bench results, task-weighted across 216 sampled executions per method over 36 distinct task-trial instances spanning airline + retail + telecom, concurrency 1/5 and fault rates 0/0.1/0.2. "Fixture" is the fixture-backed assertion score derived from public benchmark metadata, not a full benchmark-native simulator rerun; "Runtime" is the contractual runtime's final state; "Bounded recovery" counts only the recovery controller. Cost / success is reported both task-weighted and as the mean over non-empty cells. *Over-rejection* is the rate at which the runtime dead-letters a fixture-acceptable trajectory.

Method	Fixture	Runtime	Over-reject.	Dead-letter	Bounded recovery	Cost / success (weighted)
Vanilla	0.13	0.00	0.13	1.00	0.00	–
Retry-only	0.12	0.12	0.00	0.88	0.00	37.19
Schema-only	**0.15**	**0.15**	0.00	**0.85**	0.00	16.55
AdAgentFlow-RT (full)	0.15	0.15	0.00	0.85	0.00	**16.50**

6.2 Main Results

Table 1 reports the per-method averages over the full τ^3-bench main matrix, weighted by the number of tasks per cell. Vanilla tool calling produces no runtime-successful completions against the live LLM and is therefore dead-lettered for every run. The fixture and runtime success rates are reported separately: the fixture column is the fixture-backed assertion score derived from public benchmark metadata, while the runtime column is the contractual runtime's final state. Schema-only validation has the highest raw task success in this matrix, narrowly ahead of the full runtime. The full runtime's distinctive result is different: it keeps weighted cost-per-success comparable to schema-only, improves materially over retry-only, and makes contract violations plus recovery decisions explicit in the trace.

Success metric definitions. *Fixture success* is the fixture-backed assertion score (NL assertions + policy compliance) derived from public benchmark metadata rather than a full benchmark-native simulator rerun. *Runtime success* is the contractual runtime's final state (no dead-letter, no failed contract recovery). *Bounded recovery* counts only the bounded recovery controller, not the schema-only quick-repair path. In the current main matrix this metric remains zero for every evaluated configuration, so it should not be used as a headline empirical win. *Over-rejection* (false dead-letter) is the rate at which the runtime dead-letters a trajectory that the fixture would have accepted; this is the audit signal that closes the "silent failure = 0" gap.

Cost aggregation. Cost per successful task is reported in two forms to keep the aggregation rule auditable: (i) *weighted* – total cost across all cells divided by total successes, including cells with zero successes as zero-cost / zero-success contributions; (ii) *nonzero-mean* – the mean of per-cell cost-per-success, taken only over cells where successes > 0. The two values are reported side-by-side so reviewers can see the gap is small; the paper claims the *weighted* value in the prose.

Table 1 shows that *the full runtime reduces cost per successful task relative to retry-only and remains comparable to schema-only, while making runtime decisions auditable in the trace.* The full runtime is *not* the most cost-efficient configuration in this matrix; schema-only and full are within 0.3% on the weighted cost metric, and schema-only slightly leads on the nonzero-mean metric (16.55 vs.

Table 2: Runtime-stability metrics aggregated over the main matrix.

Method	Silent failures	Contract viol.	Dead-letter	Trace events / task
Vanilla	**0.00**	0.00	1.00	0.00
Retry-only	**0.00**	0.00	0.88	0.00
Schema-only	**0.00**	**1.00**	0.85	0.00
AdAgentFlow-RT (full)	**0.00**	**1.00**	0.85	**6.00**

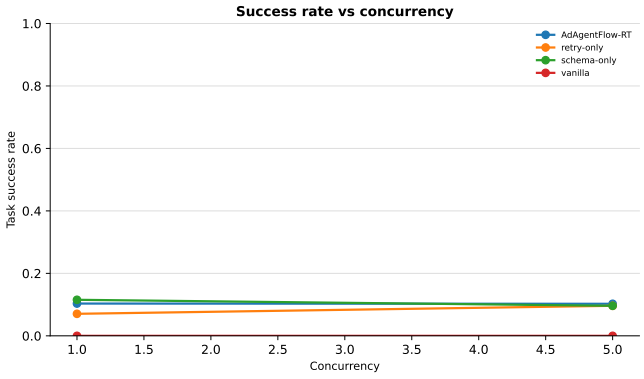

Figure 3: Task success rate by concurrency, generated from the external main-matrix summary.

16.50 weighted; 22.55 vs. 27.31 nonzero-mean). The current main matrix does *not* show non-zero bounded-recovery success for any configuration, so the differentiating empirical result is traceability plus explicit contract handling rather than a stronger recovery-success claim.

Table 2 reports the runtime-stability metrics for the same matrix. *All four configurations* report a silent-failure rate of zero on the live LLM, so this number should not be read as evidence that the full runtime uniquely prevents silent failure. The corresponding audit signal is *over-rejection* (false dead-letter): the rate at which the runtime dead-letters a trajectory the fixture would have accepted. In the current main matrix over-rejection is zero for retry-only, schema-only, and the full runtime, but vanilla reports non-zero over-rejection because it dead-letters every run while the fixture still accepts a subset of trajectories.

6.3 AgentChangeBench Coverage

AgentChangeBench rows are still executed and audited so the artifact covers both public workload families, but we do not treat the current AgentChangeBench outputs as evidence for goal-change recovery. In the current matrix the benchmark's target-shift metrics are dominated by definition effects and zero-success cells, so the earlier AgentChangeBench recovery figure and any claim based on TCRR or GSRT are intentionally omitted from the camera-ready artifact until the runtime executes genuine goal-shift workflows with benchmark-native semantics.

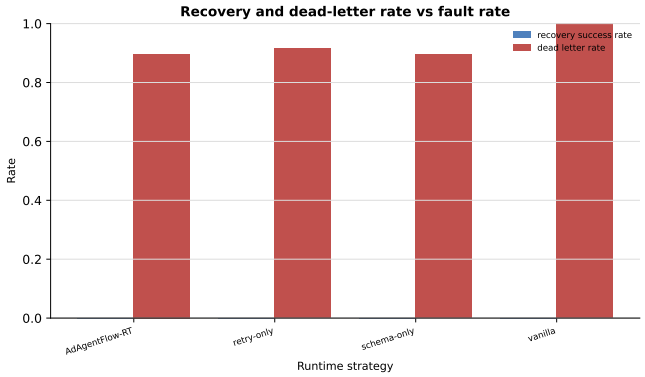

Figure 4: Recovery success and dead-letter rate under injected production faults.

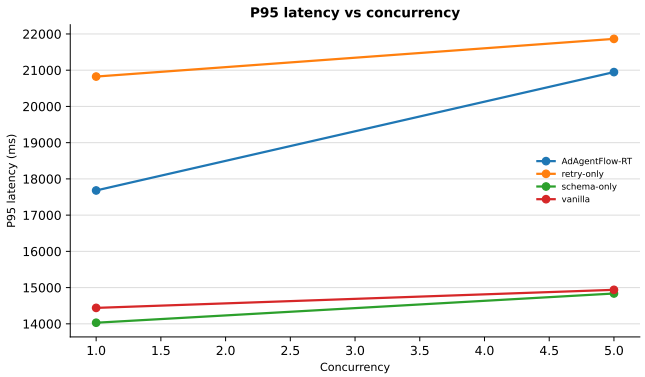

Figure 5: P95 latency by concurrency in the external main matrix.

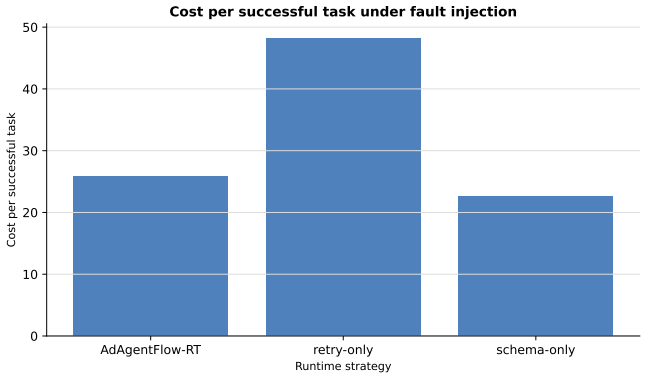

Figure 6: Nonzero-cell mean cost per successful task under fault injection.

6.4 Controlled Successful-Trajectory Fault Injection

The main matrix uses a random fault-injection schedule with a target rate. This is useful for measuring steady-state robustness but

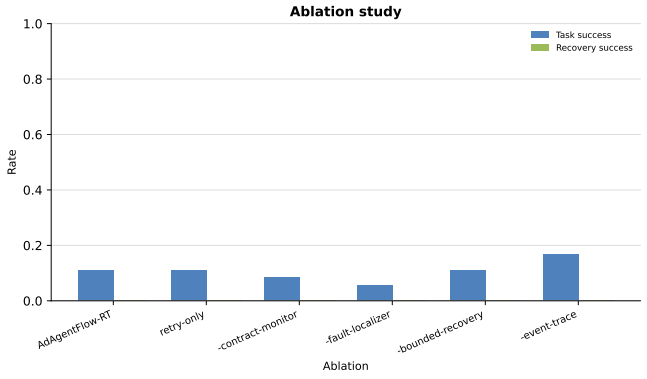

Figure 7: Ablation study over contractual runtime components. In the current matrix, recovery success remains zero across the aggregated ablation rows, so the comparison should be read mainly as a task-success and failure-containment diagnostic rather than as a recovery-success result.

does not provide strong causal attribution by fault class. The artifact therefore includes an exploratory controlled fault-injection pipeline that builds a no-fault reference set, replays each reference trajectory with a single injected stressor, and aggregates containment, runtime success, fixture success, contamination, and over-rejection outcomes.

At the current stage this pipeline should be read as an artifact component rather than as a main empirical result. The committed outputs in "paper/aamas2026/tables/controlled_fault_table.csv" are small and sparse, so they do not yet support strong per-stressor causal claims. The design note and runnable pipeline remain valuable for follow-up work because they define the reference-set construction, deterministic seeding, and aggregation protocol needed for a larger controlled study.

6.5 Ablation Study

The minimum ablation set disables one AdAgentFlow-RT component at a time. Each ablation row in the main matrix summary table is the task-weighted mean over the (telecom, fault rate 0.2, concurrency 1) cells in the same matrix; this is the only fault rate at which all four ablations and the full method were co-located. In the current aggregated outputs, recovery success remains zero for the full method and every ablation. The practical value of the ablation table is therefore diagnostic: it shows how task success and dead-letter behavior move when monitor, localizer, bounded-recovery, or event-trace components are removed, without overstating recovery-success evidence.

6.6 Reproduction and Audit

The full experiment can be re-run end-to-end on the GPU server by following the runbook. Once the GPU output directory has been copied locally, the paper-facing artifacts are regenerated with:

```
./scripts/finalize_submission.sh --local-only
```

Table 3: Evidence status for the current artifact. The status is updated by the finalization script and audit gate.

Claim	Status	Gate
Harness execution	Complete	run_real_external.py on Qwen3-8B
Runtime metrics	Complete	aggregate_suite (116 summary row
Main τ^3-bench	Complete	external audit gate
AgentChangeBench coverage	Complete	external audit gate
Paper figures	Complete	refresh_paper_artifacts

For a fresh GPU run, execute the wait script followed by the finalization script. The audit gate fails fast if matrix coverage, methods, or benchmarks are missing. Replacing the vLLM endpoint with the public OpenAI API only changes the model endpoint environment variable; the rest of the harness stays the same.

7 DISCUSSION

The contractual runtime trades additional monitoring and trace overhead for improved diagnosability and bounded recovery. The key limitation is that semantic checks require domain-specific verifiers or benchmark-specific policies. The harness is designed so stronger verifiers can be added without changing workload data.

7.1 Cost and Reliability

Contracts and traces add runtime work. The relevant tradeoff is not raw latency alone, but cost per successful task under fault injection. A retry-only strategy may appear simple, but can amplify tool and LLM calls without identifying the contaminated artifact. AdAgentFlow-RT pays monitoring overhead to make violations and recovery decisions inspectable. In the current main matrix, schema-only slightly leads raw task success, while the full runtime remains comparable on weighted cost-per-success and clearly outperforms retry-only on the same metric. The current data do not show non-zero bounded-recovery success; the differentiation is on traceability, explicit contract violations, and inspectable recovery decisions rather than on a headline recovery-success claim. This is a failure-containment claim, not a task-success leaderboard claim.

7.2 Limits of the Current Artifact

The current artifact is external-benchmark backed: all four runtime methods are driven by the `SyncLLMClient` against public benchmark task fixtures, and the main matrix is gated by an audit script that requires both external main-matrix rows and AgentChangeBench rows. The harness still includes deterministic smoke modes for development and regression checks, but those rows are not used for the paper's main claims. The remaining limitations are important: the scorer is fixture-backed rather than a full benchmark-native simulator rerun; the matrix uses a single Qwen3-8B endpoint; the empirical workflow is still prompt-centric rather than a genuine long-horizon multi-agent execution with benchmark-native tool calls and artifact propagation; several trace metrics such as mean time to detect, mean time to recover, fault propagation depth, and contaminated artifact count are placeholders unless timestamped traces and artifact-propagation

annotations are present; and the controlled fault-injection outputs are currently exploratory rather than strong causal evidence. These limits motivate a larger controlled successful-trajectory fault-injection study and a benchmark-native runtime integration as the next experiments.

7.3 Threats to Validity

Fault distributions may not match all production incidents. Semantic contracts can be incomplete or domain-biased. The task fixtures are public and external, but the execution uses one live LLM endpoint and a constrained sampling plan; stronger production claims require benchmark-native simulator reruns, additional models, larger task sweeps, independent reruns, and controlled fault injection on trajectories that first succeed without injected faults. These limits affect generality, not the artifact's audit trail: the JSONL traces, summary tables, figures, and audit report are all generated from the same external main-matrix directory.

8 RELATED WORK

This work relates to agent benchmarks, multi-agent frameworks, runtime verification, workflow orchestration, fault injection, chaos engineering, and production observability. Unlike task-only benchmark evaluations, AdAgentFlow-RT focuses on operational reliability under stress.

8.1 Agent Benchmarks

Agent benchmarks evaluate task completion, tool-use correctness, policy compliance, and goal-change behavior. This work treats tau2-bench / τ^3-bench and AgentChangeBench [2, 12] as workload providers rather than datasets to replace. The current harness preserves task fixtures and metric fields, uses assertion-based wrapper scoring, and adds runtime-stability metrics around the same tasks; a full benchmark-native simulator rerun and genuine multi-turn goal-shift execution are future validation rather than current claims. This positioning is closer to agent-evaluation work such as AgentBench and SWE-agent than to a new benchmark proposal [8, 14].

8.2 Multi-Agent Frameworks

Multi-agent frameworks provide abstractions for agent composition, tool invocation, message routing, planning, and memory [6, 13, 15]. Tool-using and self-correcting agent patterns such as Toolformer and Reflexion show the value of explicit action structure and iterative correction [10, 11]. AdAgentFlow-RT complements these capabilities with execution contracts, bounded recovery, event traces, dead-letter behavior, and stress evaluation.

8.3 Runtime Verification and Observability

Runtime verification checks whether executions satisfy specified properties [7]. AdAgentFlow-RT adapts this view to probabilistic agent workflows by making contracts executable and recovery-aware. Structured-output guardrails and schema-constrained generation systems pursue a related goal at the model-output layer [1, 5]. Production observability motivates the event-sourced trace: every violation, diagnosis, and recovery decision must be attributable to a task, run, and node, aligning with modern observability stacks such as OpenTelemetry [3].

8.4 Fault Injection

Fault injection and chaos engineering test whether a system remains reliable under controlled failures [9]. Workflow orchestrators such as Airflow provide operational scheduling abstractions but do not solve LLM-specific schema drift, semantic mismatch, or recovery-audit problems [4]. The harness applies fault-injection ideas to multi-agent runtime boundaries: tool calls, context, schema, worker execution, and duplicate delivery.

9 CONCLUSION

AdAgentFlow-RT evaluates and improves the runtime reliability of contract-aware benchmark-derived workflows. Rather than introducing a new dataset, it layers production stress, contractual monitoring, fault localization, bounded recovery, and event-sourced tracing on top of public benchmark task fixtures. The current artifact establishes the runtime kernel, stress harness, baselines, stressors, trace-derived metrics, ablations, external main-matrix results, publication-facing figures, paper tables, and audit gate. The next step is not to create more infrastructure, but to harden the empirical chain: rerun the full matrix with the repaired fault-injection path, validate against benchmark-native semantics where possible, broaden model coverage, and package a clean reproducibility artifact.

ACKNOWLEDGMENTS

The authors thank the maintainers of τ^3-bench and AgentChangeBench for the public task environments that motivate the production-stress evaluation harness in this paper. The main-matrix evidence was produced with a live Qwen3-8B endpoint served by vLLM 0.23.0 and audited from the external result directory. The artifact includes JSONL traces, summary tables, regenerated figures, and an audit report; full reproduction instructions are provided in the runbook.

REFERENCES

[1] 1rgs. 2023. Jsonformer: A Bulletproof Way to Generate Structured JSON from Language Models. https://github.com/1rgs/jsonformer.
[2] AgentChangeBench Contributors. 2026. AgentChangeBench. Public benchmark used as a supplementary goal-change workload.
[3] Cloud Native Computing Foundation. 2024. OpenTelemetry: Unified Observability for Modern Systems. *Project documentation* (2024). https://opentelemetry.io/
[4] Apache Software Foundation. 2024. Airflow: A Workflow Management Platform. In *Apache project documentation.*
[5] Guardrails AI. 2024. Guardrails: Adding Guardrails to Large Language Models. https://github.com/guardrails-ai/guardrails.
[6] LangChain Inc. 2024. LangGraph. *Software framework documentation* (2024). https://github.com/langchain-ai/langgraph
[7] Martin Leucker and Christian Schallhart. 2009. Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303.
[8] Xiao Liu, Yizhou Gu, Yicheng Xia, Yizhou Wu, Jiaqi Liu, Dong Yu, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations.*
[9] Casey Rosenthal, Nora Jones, Benjamin Daly, and Aaron Blohowiak. 2020. *Chaos Engineering: System Resiliency in Practice.* O'Reilly Media.
[10] Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL]

[11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 [cs.AI]
[12] Sierra Research. 2026. tau2-bench: Public Task Environments for Agent Evaluation. https://github.com/sierra-research/tau2-bench.
[13] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI]
[14] John Yang et al. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793 [cs.SE]
[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.